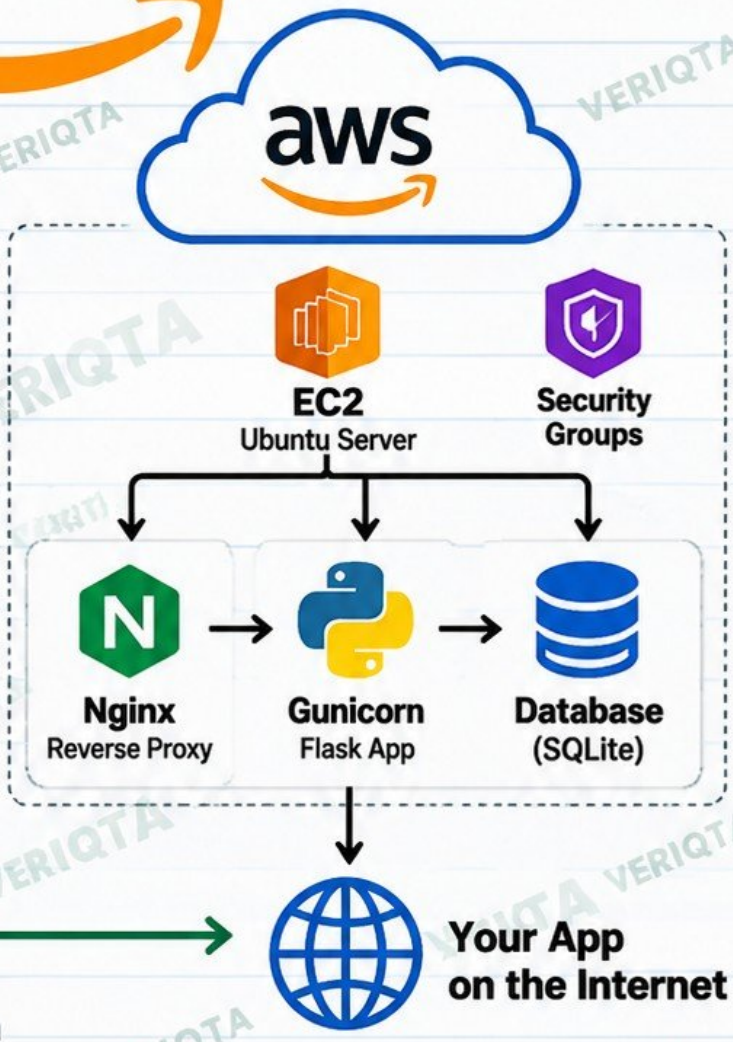
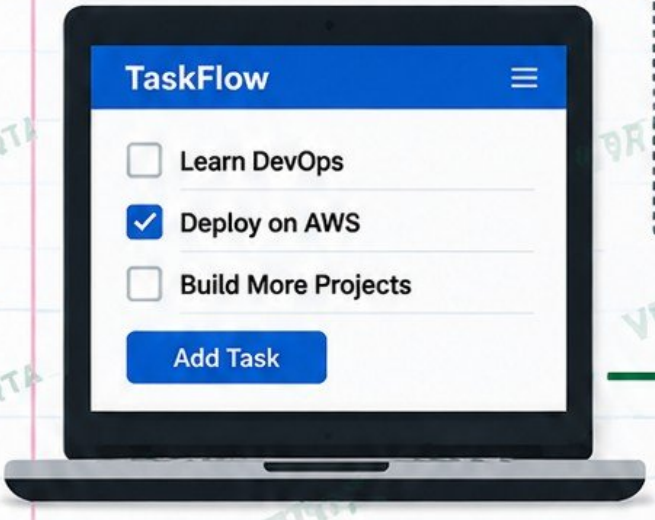
VERIQTA

# From Zero to Deployed: Your First App on AWS

- Build a Real Project
- Launch on AWS
- Configure and Secure
- Use Real DevOps Tools
- Monitor and Troubleshoot
- Update and Maintain




@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

# From Localhost to the Internet

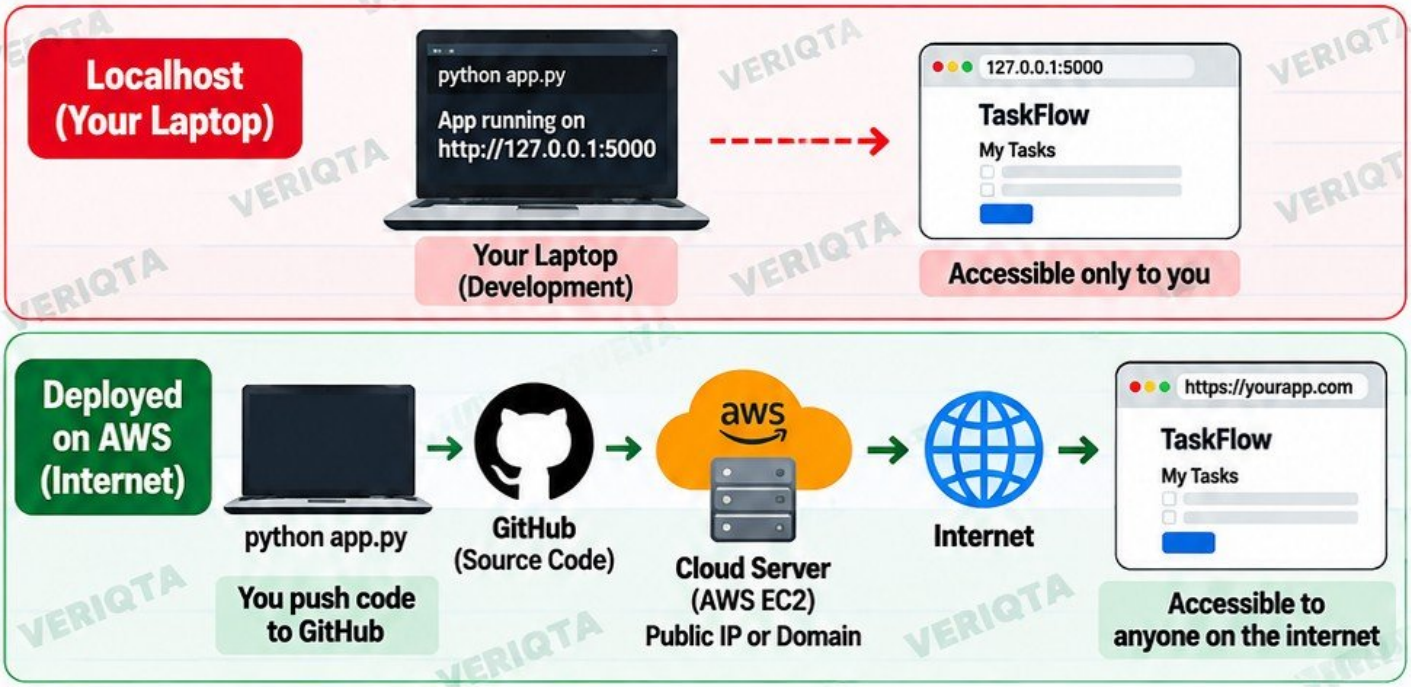
## 1. What Does It Mean to Deploy an Application?

To deploy an application means to make it **available and accessible to real users over the internet, not just on your local computer.** Deployment involves running the application on a server, configuring it, securing it, and exposing it through a public address (such as a domain name).

## 2. Localhost vs Internet Deployment

- **Localhost (your laptop)**
  - Runs only on your computer
  - Accessible only to you
  - Uses a local address like 127.0.0.1
  - Good for development and testing
- **Internet deployment (cloud server)**
  - Runs on a server in the cloud (e.g. AWS)
  - Accessible to anyone on the internet
  - Uses a public IP address or domain name
  - Requires proper configuration and security

## 3. Visual: Localhost vs Deployed Application




## Junior Engineer Insight

A running application is not necessarily a publicly reachable application. It must be deployed on a server, properly configured, and exposed to the internet with the correct network and security settings.

## Common Beginner Mistakes

- Thinking localhost is the same as a live application.
- Expecting others to access your laptop directly.
- Not configuring network and security settings.
- Forgetting to use a public IP or domain name.

## Key Takeaway

Your application goes from running on your local machine to running on a cloud server that is accessible over the internet. Deployment makes your application available to real users.

VERIQTA

# Meet the AWS Services Behind Your App

## 1. Key AWS Services

| | |
|---|---|
| **EC2** | Virtual server in the cloud that runs your application. |
| **VPC** | Isolated network where your resources live. |
| **Security Groups** | Act as a firewall to allow or block traffic. |
| **IAM** | Manages access and permissions for AWS services. |
| **CloudWatch** | Monitors your resources, logs and alerts. |
| **Route 53** | DNS service that maps your domain name to your application. |

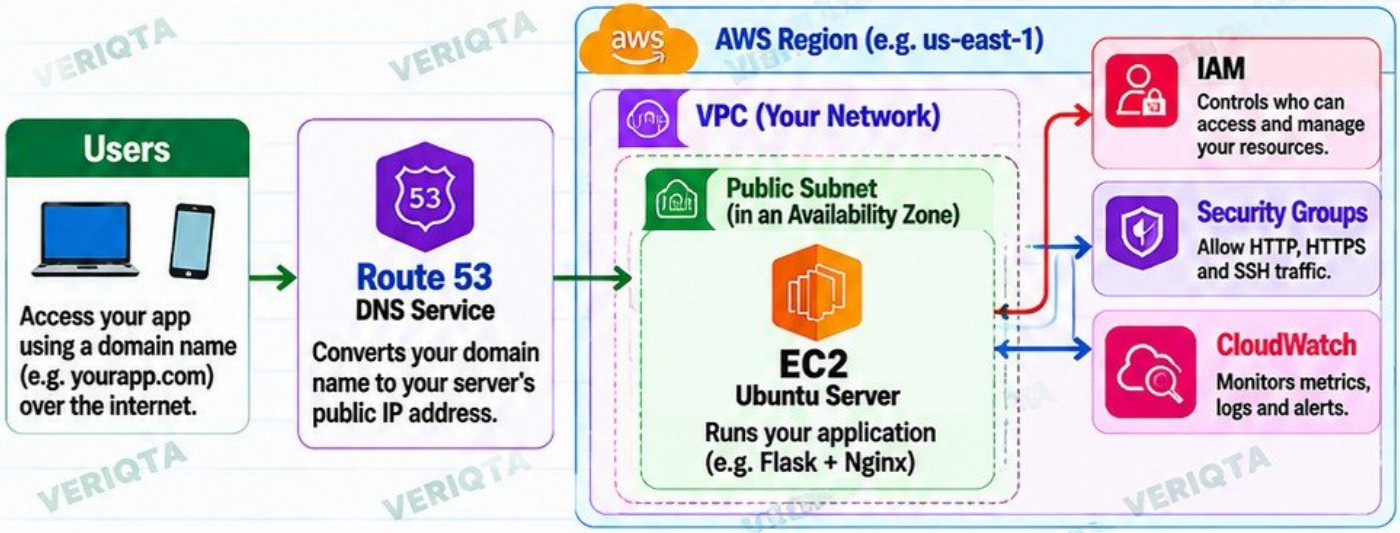
## 2. Important AWS Concepts

| | |
|---|---|
| **Regions** | • Separate geographic locations around the world.<br>• You choose a region (e.g. us-east-1). |
| **Availability Zones (AZs)** | • Multiple isolated data centres within a region.<br>• Helps with high availability. |

## 3. Which Services Do We Actually Need?

| | |
|---|---|
| ✓ **EC2** | Runs our application (Ubuntu server). |
| ✓ **VPC** | Provides the network environment. |
| ✓ **Security Groups** | Controls inbound and outbound traffic. |
| ✓ **IAM** | Manages secure access (we use IAM user). |
| ✓ **CloudWatch** | Monitors the server and application logs. |
| ✓ **Route 53** | (Optional) Maps our domain name to the server. |

## 4. How These Services Work Together




## Junior Engineer Insight

**AWS is not one server.**
**It is a collection of services that work together to run, secure, and manage your application.**

## Common Beginner Mistakes

- Thinking AWS is just EC2.
- Using the root account instead of an IAM user.
- Deploying without security groups.
- Forgetting to choose the right region.
- Not monitoring the server.
- Not using a domain name.

## Key Takeaways

- Each AWS service has a specific role.
- We need EC2, VPC, security groups, IAM and CloudWatch for this project.
- Route 53 is used if you have a domain name.
- Services work together to make your app secure, accessible and reliable.

VERIQTA

# Create Your AWS Account Safely

## 1. Root User vs IAM Identity

### Root User

- Created when you sign up for AWS
- Full access to everything
- Should only be used for account setup
- Do NOT create root access keys
- Do NOT use for daily work

### IAM Identity

- Created by you inside AWS
- Limited permissions
- Used for everyday engineering work
- Can be given specific access (least privilege)
- Supports MFA

## 2. Enable Multi-Factor Authentication (MFA)

- Adds a second layer of security when signing in.
- Even if your password is leaked, an attacker cannot access your account without the MFA code.
- Use an authenticator app (e.g. Google Authenticator, Authy) or a hardware key.




## 3. Set Up Billing Alerts

- Helps you know when your AWS costs reach a certain amount.
- Useful for beginners to avoid unexpected charges.
- You can set email alerts for any amount (e.g. $10).



## 4. Use AWS Budgets

- Track your expected and actual costs.
- Set a monthly budget (e.g. $10 or $20).
- Get alerts when you reach a percentage of your budget (e.g. 50%, 80%, 100%).
- Helps you stay in control of spending.



## 5. Choose the Right Region

- AWS has data centres around the world called Regions.
- Choose a region close to your users.
- Each region has multiple Availability Zones.
- Some services and pricing may vary by region.




## 6. Security Reminder



**Never create root access keys or use the root account for everyday engineering work. Always use an IAM user with the minimum permissions required.**

## 7. Practice: Configure an AWS Budget Alert

1. Open AWS Budgets in the AWS Console
2. Create a Budget (e.g. $10 per month)
3. Set Alert Thresholds (e.g. 50%, 80%, 100%)
4. Add Email Notifications




**Key Takeaway** Set up MFA, use an IAM user, enable billing alerts, create a budget, and choose the right region before launching any resources.

VERIQTA

# Build TaskFlow and Run It Locally

## 1. What We Are Building

- TaskFlow is a simple web application to manage a list of tasks.
- We will build it using Python and Flask.
- You will run it locally on your laptop first, then deploy it to AWS in later pages.
- This helps you confirm the application works before moving it to the cloud.

## 2. Project Structure

```
taskflow/
├── app.py              # Main application file
├── requirements.txt    # Python dependencies
├── templates/
│   └── index.html      # Web interface
└── static/
    └── style.css       # Simple styling (optional)
```

## 3. Create a Python Virtual Environment

```
# Create a project folder and enter it
mkdir taskflow
cd taskflow

# Create a virtual environment
python3 -m venv .venv

# Activate the virtual environment
source .venv/bin/activate
```

**Why use a virtual environment?**

- Keeps project dependencies separate.
- Avoids conflicts with other projects.
- Makes your project easier to manage and deploy.

## 4. Install Dependencies

```
# Install Flask and other required packages
pip install -r requirements.txt
```

**Sample requirements.txt**

```
Flask==3.0.0
gunicorn==21.2.0
```

## 5. app.py (Application Code)

app.py

```
from flask import Flask, render_template,
  request, redirect, url_for

app = Flask(__name__)

tasks = []

@app.route("/")
def index():
   return render_template("index.html", tasks=tasks)

@app.route("/add", methods= ["POST"])
def add():
   task = request.form.get("task")
   if task:
      tasks.append(task)
   return redirect(url_for("index"))

if __name__ == "__main__":
   app.run(host="127.0.0.1", port=5000, debug=True)
```

## 6. Basic Web Interface (index.html)

templates/index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>TaskFlow</title>
   <link rel="stylesheet" href="/static/style.css">
</head>
<body>
   <h1>TaskFlow</h1>
   <form action="/add" method="POST">
     <input type="text" name="task" placeholder="New task" required>
     <button type="submit">Add Task</button>
   </form
   <ul>
      {% for task in tasks %}
       <li>{{ task }}</li>
      {% endfor %}
</ul>
</body>
</html>
```

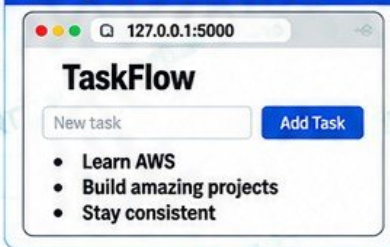
## 7. Run the Application Locally

```
# Make sure your virtual environment is active
python app.py
```

**You should see output like:**

```
* Running on http://127.0.0.1:5000
* Press CTRL+C to quit
```

## 8. Open in Your Browser




## 9. Verification

- The page loads at http://127.0.0.1:5000
- You can add a task
- The new task appears in the list
- No error messages in the terminal

**Junior Engineer Insight**

A running application on your laptop is accessible only to you. It is not publicly reachable. Deployment makes your application available to real users over the internet.

**Security Reminder**

Never create root access keys or use the root account for everyday engineering work.

**Next Step**

You have a working application. In the next pages, we will deploy it to an AWS server and make it accessible over the internet.

VERIQTA

# Understand VPCs, Subnets, and Internet Access

## 1. What Is a VPC?

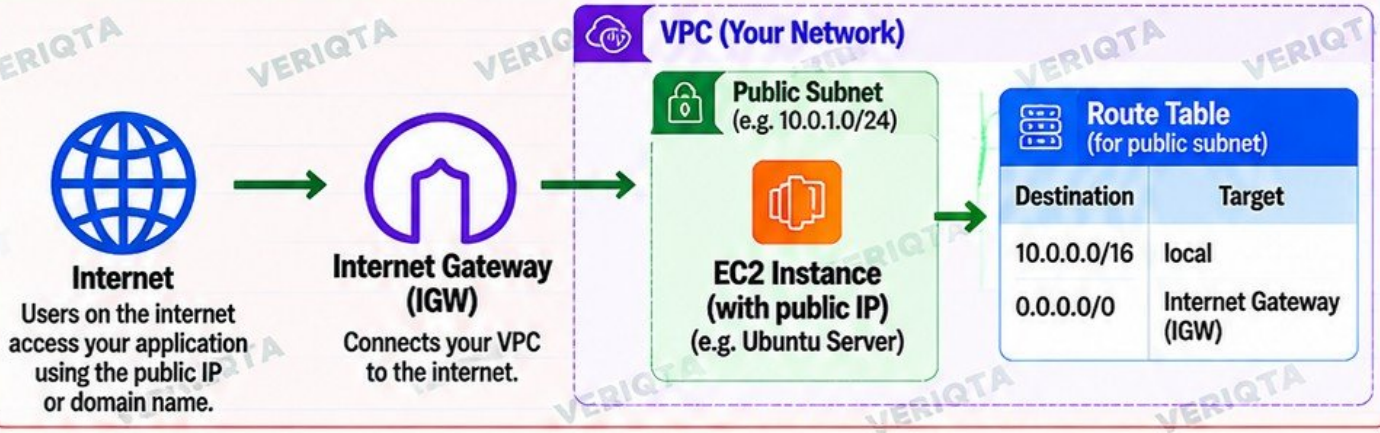
VPC

- A Virtual Private Cloud (VPC) is an isolated network in AWS where you launch your resources.
- It acts like your own private network inside the AWS cloud.
- You control the IP address range, subnets, route tables, and security.

## 2. Key Components

| | |
|---|---|
| **Internet Gateway (IGW)** | • Connects your VPC to the internet.<br>• Allows public subnets to send and receive traffic. |
| **Public Subnet** | • A subnet with a route to the internet through the IGW.<br>• Can host resources that need internet access (e.g. EC2). |
| **Route Table** | • Contains rules (routes) for where network traffic goes.<br>• Public subnet has a route to the internet (0.0.0.0/0). |
| **Public IPv4 Address** | • An IP address that is reachable over the internet.<br>• Assigned to your EC2 instance (automatically or manually). |

## 3. How They Work Together




## 4. What Makes an EC2 Instance Accessible?

- Launched in a public subnet.
- Subnet has a route to the internet (0.0.0.0/0 → IGW).
- Instance has a public IPv4 address.
- Security group allows inbound traffic (e.g. SSH on port 22, HTTP on port 80).
- Application or service is running and listening.

## 5. Common Beginner Mistake

**Assuming that placing an instance in a public subnet automatically makes it publicly accessible.**

Your instance can still be unreachable if:

- It does not have a public IP address.
- The security group blocks the required ports.
- The route table is not configured correctly.
- No application is running or listening on the port.

## Junior Engineer Insight

A public subnet only gives a path to the internet. Your instance still needs a public IP, correct security group rules, and a running application to be accessible.

## Key Takeaways

- VPC isolates your resources in AWS.
- Public subnet + route to IGW allows internet access.
- Security groups control which traffic is allowed.
- A public IP lets people on the internet reach your instance.
- All components must be correctly configured.

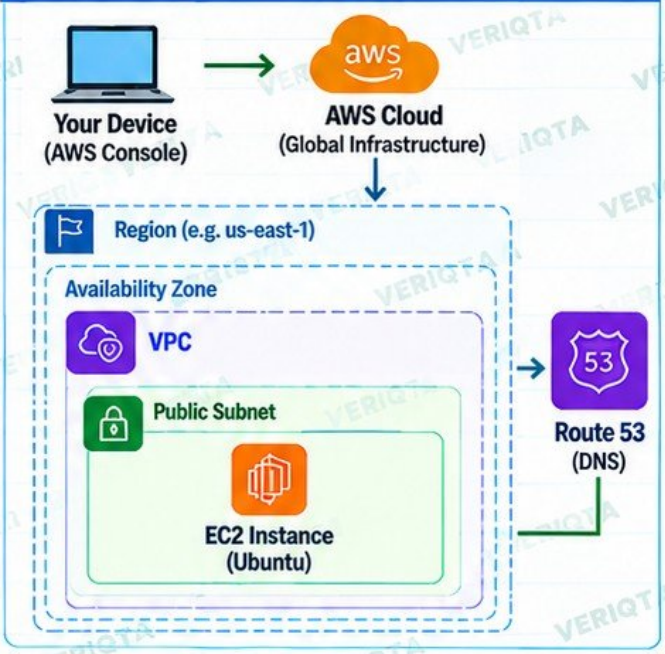
VERIQTA

# Launch Your First Ubuntu EC2 Instance

## 1. Key Concepts

| | |
|---|---|
| **AMIs** | • Preconfigured templates for your instances.<br>• Ubuntu is one of the options. |
| **Instance Types** | • Define CPU, memory and storage capacity.<br>• e.g. t2.micro (Free Tier). |
| **Key Pairs** | • Secure SSH access to your instance.<br>• You create a key pair when launching. |
| **Storage Volumes** | • EBS provides persistent storage.<br>• Default size is usually 8–30 GB. |
| **Public IP Address** | • Makes your instance reachable from the internet.<br>• Can be dynamic or elastic. |
| **Instance States** | • Pending, Running, Stopped, Stopping, Terminated.<br>• Running means it's ready to use. |

## 2. How It All Works Together




## 3. Launch the Instance (Step by Step)

**1 Choose AMI**
- Ubuntu Server 22.04 LTS
- Free Tier eligible

**2 Select Instance Type**
t2.micro
- t2.micro
- 1 vCPU, 1 GB RAM

**3 Create Key Pair**
- Name: taskflow-key
- Download .pem file
- Keep it safe

**4 Configure Storage**
- 8–30 GB (default)
- Use gp3 (default)
- Enable termination protection (optional)

**5 Configure Network**
- Public subnet
- Auto-assign public IP
- Security group (ports 22, 80, 443)

**6 Launch**
- Review settings
- Click Launch
- Wait for running state

## 4. Practice

- [ ] Launch a small Ubuntu instance using the steps above.
- [ ] Record your instance ID (e.g. i-0abc123def456).
- [ ] Record your public IP address (e.g. 3.12.45.67).
- [ ] Confirm the instance is in the Running state.

**Example Output**

Instance ID:
`i-0abc123def456`

Public IP:
`3.12.45.67`

State:
**running**

### Production Awareness

- Free Tier eligibility depends on your account and usage.
- EC2, public IPv4 addresses, and storage can incur charges.
- Always check the AWS pricing page and set up billing alerts.

### Junior Engineer Reminder

- Launching an instance is just the beginning. You still need to secure it, connect to it, and deploy your application.

VERIQTA

# Secure Your Server With Security Groups

## 1. What Is a Security Group?

Security Group

- A security group acts as a virtual firewall for your EC2 instance.
- It controls which traffic is allowed in (inbound) and out (outbound).
- It operates at the instance level.
- It is stateful, which means if you allow incoming traffic, the return traffic is automatically allowed.

## 2. Key Terms

| | |
|---|---|
| Inbound Rule | Controls traffic coming to your instance. |
| Outbound Rule | Controls traffic leaving your instance. |
| TCP Port | A numbered endpoint used by applications (e.g. 22, 80, 443). |
| Source IP Range | Defines where the traffic is allowed from (e.g. your IP or anywhere: 0.0.0.0/0). |
| Stateful | Return traffic is automatically allowed if the inbound traffic is permitted. |

## 3. Recommended Inbound Rules for This Project

| Type | Protocol | Port | Source | Purpose |
|---|---|---|---|---|
| SSH | TCP | 22 | Your IP address (e.g. 203.0.113.25/32) | Allow SSH access from your trusted IP only. |
| HTTP | TCP | 80 | 0.0.0.0/0 (Anywhere) | Allow public web traffic. |
| HTTPS | TCP | 443 | 0.0.0.0/0 (Anywhere) | Allow secure web traffic. |

## 4. Visual: How the Rules Work




## 5. Outbound Rules

| Type | Protocol | Port | Destination | Purpose |
|---|---|---|---|---|
| All traffic | All | All | 0.0.0.0/0 | Default rule (usually allowed). Lets your instance access the internet (e.g. updates, packages). |

## 6. Security Reminder

- Never expose SSH (port 22) to the entire internet (0.0.0.0/0) without understanding the risk.
- Use your current public IP or a narrow range.
- Consider using a VPN or AWS Systems Manager Session Manager for better security.

## 7. Check Your Security Group

- Confirm inbound rules for ports 22, 80, and 443.
- Verify the source IP for SSH is restricted to your IP.
- Leave outbound rule as default (allow all).
- Test the rules after launching your instance.

## Junior Engineer Insight

A security group only controls network traffic. It does not replace proper server hardening, strong SSH keys, or user management. Always use the principle of least privilege.

PAGE 08

VERIQTA

# Connect to EC2 Using SSH

## 1. What Is SSH?

SSH

- Secure Shell (SSH) is a secure protocol used to connect to a remote server.
- It encrypts your connection.
- You use a private key to authenticate instead of a password.
- It allows you to run commands, manage files, and configure your server securely.

## 2. Key Concepts

| | |
|---|---|
| **Private Key** | • A file (e.g. taskflow.pem) used to prove your identity.<br>• Keep it safe and never share it. |
| **Username** | • For Ubuntu AMIs, the default username is ubuntu.<br>• Other AMIs may use ec2-user or another username. |
| **Host Address** | • The public IP address or domain name of your EC2 instance.<br>• Example: 3.12.45.67 |
| **Authentication** | • SSH uses your private key to authenticate you.<br>• No password is required when using key-based authentication. |

## 3. How SSH Connection Works




**Important**

- SSH uses port 22.
- Your security group must allow inbound traffic on port 22 from your IP address.
- You must have the correct private key and username.

## 4. Commands to Connect

1. Set correct permissions on your private key

```bash
chmod 400 taskflow.pem
```

2. Connect to your EC2 instance

```bash
ssh -i taskflow.pem ubuntu@SERVER_IP
```

## 5. Example

```bash
chmod 400 taskflow.pem
ssh -i taskflow.pem ubuntu@3.12.45.67
```

```bash
The authenticity of host '3.12.45.67 (3.12.45.67)'
can't be established.
Are you sure you want to continue connecting (yes/no)? yes
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux x86_64)
ubuntu@ip-172-31-12-34:~$
```

## 6. Common Troubleshooting

| Error | Possible Cause | Solution |
|---|---|---|
| Permission denied (publickey) | Wrong key, wrong username, or incorrect key permissions. | • Use correct key.<br>• Check username (ubuntu).<br>• Run chmod 400 taskflow.pem |
| Connection timed out | Security group blocking port 22 or wrong IP address. | • Allow port 22 from your IP.<br>• Check the instance public IP. |
| No such file or directory | Key file path is incorrect. | • Use the correct path to your .pem file. |
| ssh: connect to host ... port 22: Connection refused | Instance not running or SSH blocked by security group. | • Ensure the instance is running.<br>• Check security group rules. |

## 7. Security Best Practices

- Keep your private key secure and never share it.
- Restrict SSH access to your IP address.
- Do not use the root user for SSH.
- Consider using a bastion host or AWS Systems Manager Session Manager for better security.

**Junior Engineer Insight**

- SSH gives you command-line access to your server.
- Always verify your key permissions and username.
- If SSH is not working, check security group rules firt.
- Error messages usually tell you exactly what is wrong.

VERIQTA

# Prepare Ubuntu for Application Hosting

## 1. What We Need and Why

| | |
|---|---|
| **Package Updates** | • Keep the server secure and up to date.<br>• Get the latest bug fixes and features. |
| **Python** | • Required to run the application.<br>• We will use a virtual environment. |
| **Git** | • Used to clone or download your code.<br>• Helps with version control. |
| **Nginx** | • A web server and reverse proxy.<br>• Helps serve your application. |
| **Linux Users** | • Run the application as a normal user (not root) for security.<br>• Keep the system organized. |
| **Application Directories** | • Store your application files in a dedicated directory (e.g. /opt/taskflow).<br>• Keep a clean and structured setup. |

## 2. Package Updates

```bash
sudo apt update
sudo apt upgrade -y
```

This installs the latest security updates and improves stability.

## 3. Install Required Packages

```bash
sudo apt install -y python3 python3-venv python3-pip git nginx
```

This installs Python, virtual environment support, pip, Git, and Nginx.

## 4. Verify Installed Versions

```bash
python3 --version
```
Python 3.x.x

```bash
pip3 --version
```
pip 24.x.x

```bash
git --version
```
Git 2.x.x

```bash
nginx -v
```
Nginx 1.x.x

## 5. Create Application Directory and User

```bash
sudo useradd -m -s /bin/bash taskflow
sudo mkdir -p /opt/taskflow
sudo chown -R taskflow:taskflow /opt/taskflow
```

- Creates a dedicated user (taskflow).
- Creates an application directory (/opt/taskflow).
- Sets the correct ownership.

## 6. Set Up Python Virtual Environment

```bash
cd /opt/taskflow
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
```

A virtual environment keeps your application dependencies separate from the system Python.

## 7. Check Your Setup

| Item | Command | Expected Output |
|---|---|---|
| Python | `python3 --version` | Python 3.x.x |
| pip | `pip3 --version` | pip 24.x.x |
| Git | `git --version` | git 2.x.x |
| Nginx | `nginx -v` | nginx 1.x.x |
| Virtual env | `source .venv/bin/activate` | (.venv) in prompt |
| App directory | `ls -ld /opt/taskflow` | Owned by taskflow user |

## 8. Junior Engineer Tip

- Avoid installing application dependencies globally.
- Do not run your application as root.
- Use a dedicated user and a virtual environment.
- Keep your server organized with clear directories.
- This improves security, stability, and makes deployment easier.

VERIQTA

# Move Your Application From GitHub to EC2

## 1. What We Are Doing and Why

**Git Repository**
- Stores your application code.
- Allows version control and collaboration.
- You will clone the repository to your EC2 instance and run the application.

**EC2 Instance**
- A remote server where your application will run.
- You copy (clone) the code, install dependencies, and start the application.

**Reproducible Deployment**
- Using Git and a requirements file ensures you can deploy the same application again on any server.

## 2. Key Concepts

| | |
|---|---|
| **Git Clone** | Downloads a copy of the repository to your EC2 instance. |
| **Branches** | Usually you deploy from the main (or a specific) branch. |
| **Dependencies** | Python packages listed in requirements.txt. |
| **Reproducibility** | Anyone can set up the same application using the same steps and files. |

## 3. Clone the Repository on EC2

```bash
# Go to your application directory
cd /opt

# Clone your repository
git clone https://github.com/your-username/taskflow.git

# Enter the project directory
cd taskflow

# Check the status
git status
```

### Notes
- Replace your-username with your actual GitHub username.
- Use the correct repository URL (HTTPS or SSH).
- If you are using a private repository, make sure your EC2 instance has access (SSH key or GitHub token).
- By default, this clones the main branch.

## 4. Check the Cloned Files

```bash
# List the files in the project directory
ls -la

# Example output
app.py  requirements.txt  templates/  static/
```

### Verification
- You should see your application files (e.g. app.py, requirements.txt).
- Templates and static folders (if any) should be present.
- This confirms the code has been successfully cloned to EC2.

## 5. Install Python Dependencies

```bash
# (Optional) Create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install required packages
pip install -r requirements.txt
```

### What This Does
- Creates a virtual environment (recommended).
- Installs all required Python packages listed in requirements.txt.
- Keeps your application dependencies separate from the system Python.

## 6. Verify Installed Dependencies

```bash
# Check installed packages
pip list

# Or check a specific package (example)
pip show flask
```

### Verification
- You should see a list of installed packages (e.g. Flask, gunicorn, etc.).
- This confirms your application dependencies are installed on EC2.

## 7. Junior Engineer Tips

- Always use Git to move code, not manual file copying.
- Keep your requirements.txt updated.
- Use a virtual environment, not global installation.
- Deploy from a specific branch when needed.
- Keep your repository organized and clean.

VERIQTA

# Run Flask With Gunicorn

## 1. Why Not Use Flask's Development Server?

Flask

- Flask's built-in development server is designed for development only.
- It handles only one process and is not optimized for production traffic.
- It is not as secure, reliable, or performant as a production WSGI server like Gunicorn.
- For production, use Gunicorn behind a reverse proxy (e.g. Nginx).

## 2. What Is Gunicorn?

Gunicorn

- Gunicorn is a production-ready WSGI HTTP server for Python applications.
- It runs multiple worker processes to handle requests.
- It is stable, fast, and widely used for Flask applications.
- It should bind to 127.0.0.1 (localhost) and be used behind Nginx in production.

## 3. Install Gunicorn

```bash
# Activate your virtual environment (if not already active)
source .venv/bin/activate

# Install Gunicorn
pip install gunicorn

# Verify installation
gunicorn --version
```

### What This Does

- Installs Gunicorn in your project virtual environment.
- Keeps your server dependencies separate from the system Python.
- You should always use a virtual environment for your application.

## 4. Run Flask Application With Gunicorn

```bash
# Make sure you are in your project directory
cd /opt/taskflow

# Run with Gunicorn (example)
gunicorn --bind 127.0.0.1:8000 app:app

# With multiple workers (adjust based on your server)
gunicorn --werkers 3 --bind 127.0.0.1:8000 app:app
```

## 5. Command Explanation

```
gunicorn --bind 127.0.0.1:8000 app:app
```

| | |
|---|---|
| gunicorn | Starts the Gunicorn server. |
| --bind 127.0.0.1:8000 | Binds to localhost on port 8000 (only accessible from this server). |
| app:app | Python file and Flask application object (e.g. app.py with app = Flask(__name__)). |
| --workers 3 | Runs 3 worker processes to handle requests. |

## 5. Verify It Is Running

```bash
# Check application health endpoint
curl http://127.0.0.1:8000/health
# You should see a response like:
{"status":"healthy"}
```

### Expected Result

- You receive a valid HTTP response (e.g. JSON or text).
- This confirms Gunicorn is running and your application is working on localhost.
- You can also check with: ss -tlnp | grep 8000

## 6. Common Failures and Fixes

| Error Message | Possible Cause | Solution |
|---|---|---|
| ModuleNotFoundError | Missing Python package. | Install dependencies: pip install -r requirements.txt |
| Failed to find application | Incorrect app:app reference or wrong directory. | Check your file name and application object. |
| Address already in use | Port 8000 is already in use. | Use a different port or stop the existing process. |
| Worker failed to boot | Application import error. | Check logs, fix the import error, and ensure all dependencies are installed. |

## 7. Junior Engineer Tips

- Always use Gunicorn (or another WSGI server) for production, not Flask's development server.
- Bind to 127.0.0.1 and use Nginx as a reverse proxy.
- Start with 2–4 workers, then adjust based on your server size.
- Check logs if the application fails to start.
- Keep your project structure clean and ensure all dependencies are installed.

VERIQTA

# Keep Your App Running With systemd

## 1. What Is systemd?

systemd

- systemd is the service manager used by most modern Linux distributions (including Ubuntu).
- It can run your application in the background as a service.
- It automatically starts the service at boot, restarts it on failure, and helps you manage and view logs.

## 2. Key Concepts

| | |
|---|---|
| **Service** | A long-running process managed by systemd. |
| **Service Unit** | A configuration file that tells systemd how to run your application. |
| **Automatic Startup** | The service starts automatically when the server boots. |
| **Restart Behavior** | systemd can restart your service automatically if it fails. |
| **Application Logs** | Use journalctl to view real-time and historical logs. |

## 3. Create a systemd Service Unit for TaskFlow

```bash
sudo nano /etc/systemd/system/taskflow.service
# Example content
[Unit]
Description=TaskFlow Flask Application
After=network.target

[Service]
User=taskflow
WorkingDirectory=/opt/taskflow
ExecStart=/opt/taskflow/.venv/bin/gunicorn --bind 127.0.0.1:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

## 4. Useful Commands

```bash
# Reload systemd after creating the unit file
sudo systemctl daemon-reload
```

```bash
# Start the service
sudo systemctl start taskflow
```

```bash
# Check service status
systemctl status taskflow
```

```bash
# View recent logs
journalctl -u taskflow -n 50
```

## 5. Enable and Verify Automatic Startup

```bash
# Enable the service to start on boot
sudo systemctl enable taskflow
# Start the service
sudo systemctl start taskflow
# Check status
systemctl status taskflow
```

## Expected Result

- The service should be active (running).
- It will start automatically after a server reboot.
- If the application crashes, systemd will restart it automatically.
- You can view logs using journalctl.

## 6. Failure Lab: Stop, Observe, and Recover

```bash
# Stop the service
sudo systemctl stop taskflow
# Check status (should show inactive)
systemctl status taskflow
# Try accessing the application (should fail)
curl http://127.0.0.1:8000/health
# Start the service again
sudo systemctl start taskflow
# Verify it is running
systemctl status taskflow
# Check that the application responds
curl http://127.0.0.1:8000/health
```

## 7. Common Issues and Fixes

| Issue | Solution |
|---|---|
| Service fails to start | Check logs: journalctl -u taskflow -n 50 |
| Application import error | Ensure you are in the correct directory and virtual environment. |
| Permission denied | Use the correct user and file permissions. |
| Port already in use | Check if another process is using the port or change the port. |
| Service keeps restarting | Check application logs for errors and fix missing dependencies. |

## 8. Junior Engineer Tips

- Always use a dedicated Linux user for your application.
- Keep your service unit file simple and clear.
- Use Restart=always for production.
- Check logs regularly (journalctl -u taskflow).
- If the service fails, read the logs to find the real error.
- Keep your code and environment organized.

VERIQTA

# Make Your App Public With Nginx

## 1. What and Why?

Nginx (Reverse Proxy)

- Nginx receives HTTP requests from the internet on port 80.
- It forwards the requests to Gunicorn running on localhost (127.0.0.1:8000).
- Gunicorn remains bound to localhost for security and is not directly exposed to the internet.
- This setup separates public traffic (Nginx) from the application server (Gunicorn + Flask).

## 2. How It Works (Architecture)




## 3. Configure Nginx

```nginx
sudo nano /etc/nginx/sites-available/taskflow
server {
  listen 80;
  server_name _;  # or your domain name
  location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

## 4. Enable the Configuration

```bash
# Enable the site
sudo ln -s /etc/nginx/sites-available/taskflow \
/etc/nginx/sites-enabled/

# Test Nginx configuration
sudo nginx -t

# Reload Nginx
sudo systemctl reload nginx
```

## 5. Visit Your Application

```bash
# Use your EC2 public IP address
http://YOUR_EC2_PUBLic_IP
# Or if you have a domain name
http://your-domain.com
```

## 6. End-to-End Flow




## Key Points

- Nginx handles public traffic on port 80.
- It acts as a reverse proxy and forwards requests to Gunicorn.
- Gunicorn listens only on 127.0.0.1:8000.
- This keeps your application server private and more secure.
- Always use Nginx (or a similar reverse proxy) in production.

## 7. Common Issues and Fixes

| Error | Possible Cause | Solution |
|---|---|---|
| 502 Bad Gateway | Gunicorn not running or wrong port. | Check Gunicorn service and port 8000. Run: systemctl status taskflow |
| Connection refused | Nginx cannot reach Gunicorn. | Verify Gunicorn is running on 127.0.0.1:8000. |
| Application not loading | Incorrect proxy configuration. | Check the Nginx config and run sudo nginx -t. |
| Permission denied | Nginx cannot read config file. | Check file permissions and location. |

## 8. Junior Engineer Tips

- Keep Gunicorn bound to localhost.
- Use Nginx as a reverse proxy for all public traffic.
- Check Nginx and Gunicorn logs if you see errors.
- Test your configuration after every change using sudo nginx -t.
- A 502 error usually means Nginx cannot connect to your application.

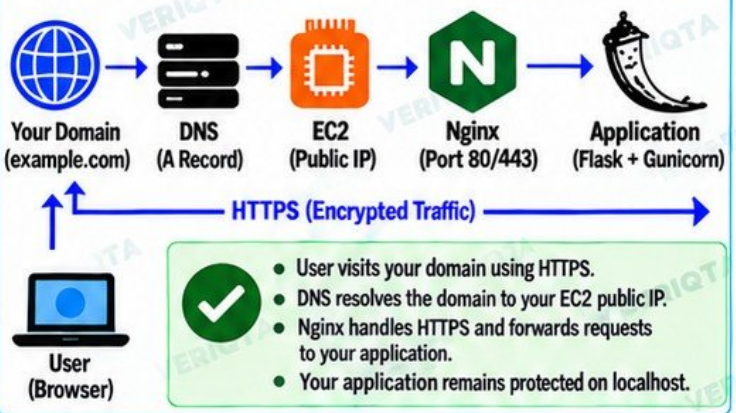
VERIQTA

# Connect a Domain and Enable HTTPS

## 1. Key Concepts

| | |
|---|---|
| **Domain** | Your domain name (e.g. example.com) points to your server using DNS. |
| **DNS A Record** | Maps your domain to your server's public IPv4 address. |
| **Stable IP** | Consider using an Elastic IP to keep the same IP address. |
| **TLS Certificate** | A digital certificate that encrypts traffic between the user and your server. |
| **HTTPS** | Secure version of HTTP using TLS. |
| **Certbot** | A tool to easily obtain and manage TLS certificates from Let's Encrypt. |

## 2. End-to-End Flow




- User visits your domain using HTTPS.
- DNS resolves the domain to your EC2 public IP.
- Nginx handles HTTPS and forwards requests to your application.
- Your application remains protected on localhost.

## 3. Point Your Domain to Your Server

```bash
# Get your EC2 public IP
curl -s ifconfig.me

# In your domain registrar DNS settings
# Create an A record:
Name:   @     (or your subdomain, e.g. app)
Type:   A
Value:  YOUR_EC2_PUBLIC_IP
TTL:    300 (or default)
```

## 4. Install Certbot and Obtain a Certificate

```bash
# Update package list
sudo apt update

# Install Certbot and Nginx plugin
sudo apt install certbot python3-certbot-nginx -y

# Obtain and configure certificate for your domain
sudo certbot --nginx -d example.com -d www.example.com
```

## 5. Verify HTTPS is Working

```bash
# Check certificate details
sudo certbot certificates

# Test your domain (replace with your domain)
curl -I https://example.com

# Open in browser
https://example.com
```

## 6. Nginx Configuration (HTTPS)

```nginx
server {
    listen 80;
    server_name example.com www.example.com;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name example.com www.example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

## 7. Common Issues and Fixes

| Issue | Possible Cause | Solution |
|---|---|---|
| Domain not resolving | DNS A record not set or propagated | Check DNS settings and wait for propagation (can take minutes or hours). |
| Certificate request fails | Nginx not pointing to server or ports blocked | Ensure domain points to your EC2 IP and ports 80 and 443 are open. |
| 502 Bad Gateway | Application not running or wrong proxy settings | Check Gunicorn is running on 127.0.0.1:8000 and Nginx config. |
| SSL certificate not trusted | Incorrect certificate setup | Use Certbot and ensure correct domain names. |

## 8. Security Reminder

- Do not claim your site is secured just because port 443 is open.
- HTTPS requires a valid TLS certificate.
- Keep your server, Nginx, and application updated.
- Use strong domain registrar security (2FA).
- Consider using an Elastic IP for a stable address.
- Regularly renew certificates (Certbot does this automatically).

## 9. Junior Engineer Tips

- Use a custom domain instead of the raw EC2 public IP.
- Verify both example.com and www.example.com work.
- Keep HTTP redirected to HTTPS.
- Check logs if you see errors: `sudo journalctl -u nginx -n 50`
- Test from a browser and using `curl`.

VERIQTA

# Protect Application Secrets and Data

## 1. Key Concepts

**Environment Variables**
- Store configuration outside your source code.
- Keep sensitive values out of your code and version control.

**Sensitive Configuration**
- Includes database credentials, API keys, and secret tokens.
- Use secure storage and access controls.

**IAM Roles**
- Allow your EC2 instance to access AWS services securely without hardcoding credentials.

**AWS Systems Manager Parameter Store**
- Store sensitive values securely.
- Retrieve them at runtime with IAM permissions.

**File Permissions**
- Restrict access to configuration files and private keys.
- Use the principle of least privilege.

## 2. How It Works (Overview)




- Your application retrieves secrets at runtime.
- Secrets are stored securely in Parameter Store.
- No sensitive data in your source code.
- Access is controlled using IAM roles and permissions.

## 3. Example: Using Environment Variables

```bash
# Create a .env file (keep this file out of GitHub)
DB_URL=postgresql://user:password@db:5432/app
API_KEY=your_api_key_here
FLASK_ENV=production

# Load environment variables in your application (Python)
import os
db_url = os.getenv("DB_URL")
api_key = os.getenv("API_KEY")
```

## 4. Store Secrets in AWS Systems Manager Parameter Store

```bash
# Store a secret value
aws ssm put-parameter \
   --name "/taskflow/db_url" \
   --value "postgresql://user:password@db:5432/app" \
   --type SecureString

# Retrieve the secret value
aws ssm get-parameter \
   --name "/taskflow/db_url" \
   --with-decryption \
   --query "Parameter.Value" \
   --output text
```

## 5. Use IAM Role for EC2 Access

```bash
# Your EC2 instance should have an IAM role attached
# with permission to read from Parameter Store.

# Example IAM policy (read access to a specific path)
{
  "Version": "2012-10-17",
  "Statement": [{
     "Effect": "Allow",
     "Action": ["ssm:GetParameter", "ssm:GetParameters"],
     "Resource": "arn:aws:ssm:REGION:ACCOUNT_ID:parameter/taskflow/*"
} }]
```

## 6. File Permissions

```bash
# Set restrictive permissions on sensitive files
chmod 600 .env
chmod 600 ~/.ssh/id_rsa
chmod 644 app.py

# Check permissions
ls -l .env
ls -l ~/.ssh/id_rsa
```

## 7. Best Practices

- Keep all sensitive data out of your source code.
- Use environment variables or Parameter Store.
- Use IAM roles instead of long-term AWS credentials.
- Restrict file permissions.
- Rotate secrets regularly.
- Give only the minimum permissions required.

## 8. Security Reminder

- Never commit passwords, API keys, or private SSH keys to GitHub.
- Do not share your .env files.
- Use IAM roles and Parameter Store for AWS credentials.
- Remove sensitive data from logs.
- Regularly rotate and revoke unused keys.

## 9. Junior Engineer Tips

- Keep a separate .env file and add it to .gitignore.
- Use meaningful environment variable names.
- Use Parameter Store for production.
- Check your file permissions.
- Review your IAM role permissions regularly.

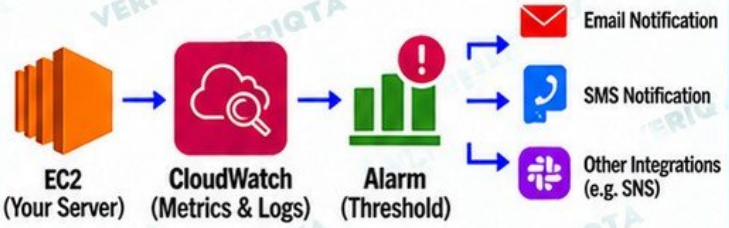
VERIQTA

# Monitor Your EC2 Server With CloudWatch

## 1. Key Concepts

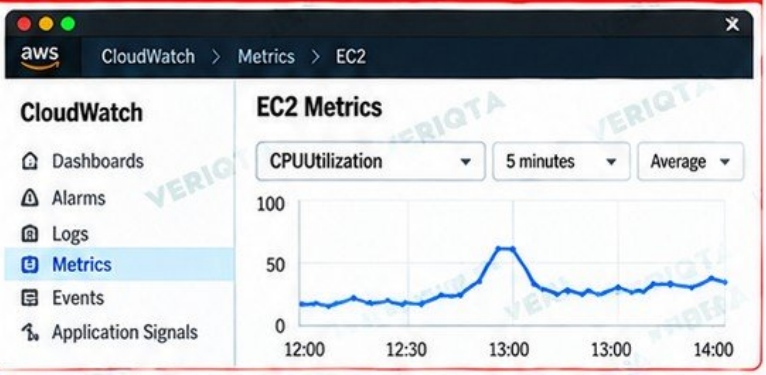
- **CPU Utilization** — Shows how much of the CPU is being used (high usage may indicate high traffic or a problem).
- **Instance Status Checks** — Monitors the health of your EC2 instance (system and instance level checks).
- **Memory Monitoring** — Not enabled by default. Install and configure the CloudWatch agent to monitor RAM usage.
- **Alarms** — Sends notifications when a metric crosses a threshold (e.g. high CPU).
- **Log Collection** — Collects and stores application and system logs using CloudWatch Logs.

## 2. How It Works (Overview)

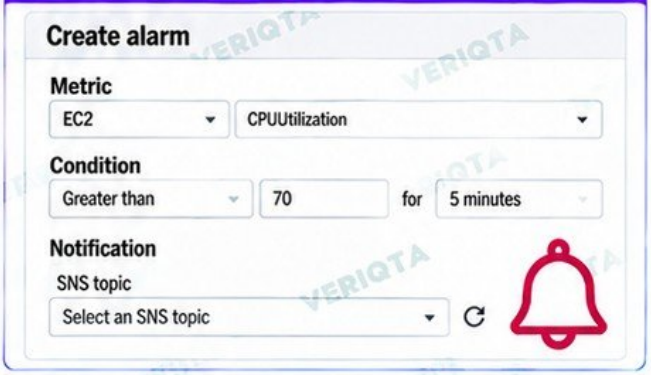



- EC2 sends metrics and logs to CloudWatch.
- You create alarms based on important metrics.
- When a threshold is crossed, you receive a notification.
- This helps you detect and respond to issues quickly.

## 3. View EC2 Metrics in CloudWatch




## 4. Create a CloudWatch Alarm




## 5. Enable Memory Monitoring (CloudWatch Agent)

```bash
# Install the CloudWatch agent
sudo apt update
sudo apt install amazon-cloudwatch-agent -y

# Configure and start the agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
sudo systemctl enable amazon-cloudwatch-agent
sudo systemctl start amazon-cloudwatch-agent
```

## 6. View Application Logs in CloudWatch

```bash
# View system logs
sudo journalctl -u taskflow -n 50

# Install and configure CloudWatch Logs agent
sudo apt install amazon-cloudwatch-agent -y
# Follow application logs (example)
sudo tail -f /var/log/nginx/access.log
```

## 7. Practice: Set Up a Useful Alert

- View your EC2 CPU utilization in CloudWatch.
- Create an alarm for CPU > 70% for 5 minutes.
- Configure an SNS notification (email or SMS).
- Test the alarm by generating CPU load (e.g. using stress).
- Confirm that you receive the notification.

## 8. Junior Engineer Insight

- A healthy EC2 instance does not guarantee a healthy application.
- Your application can be down even if CPU, memory, and status checks look normal.
- Monitor both infrastructure metrics and application logs.
- Set meaningful alarms based on your application's needs.

## 9. Common Issues and Fixes

| Issue | Possible Cause | Solution |
|---|---|---|
| No metrics visible | CloudWatch agent not installed (for memory) | Install and configure the CloudWatch agent. |
| Alarm not triggering | Threshold too high or wrong metric | Check metric, threshold, and evaluation period. |
| No notifications | SNS topic not configured or email not confirmed | Create SNS topic and confirm your email. |
| High CPU alerts often | Normal traffic or resource limits | Adjust threshold or scale your instance. |

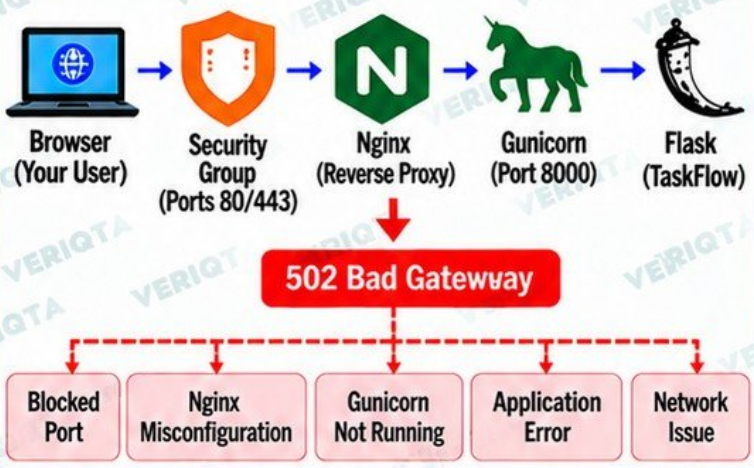
VERIQTA

# Troubleshoot a Broken AWS Deployment

## 1. Investigation Steps (Evidence-Based)

| | | |
|---|---|---|
| 1 | Browser Errors | • Check the error message (e.g. 502).<br>• Confirm the domain/IP and HTTPS. |
| 2 | Security Groups | • Verify inbound rules (80, 443, 22).<br>• Confirm source IP and port access. |
| 3 | Nginx Logs | • Check /var/log/nginx/error.log.<br>• Look for upstream errors (502). |
| 4 | systemd | • Check service status and restart.<br>• View recent logs for failures. |
| 5 | Application Logs | • Check application logs for errors.<br>• Look for exceptions or crashes. |
| 6 | Network Checks | • Test local connectivity (curl).<br>• Verify ports and process status. |

## 2. Architecture and Where Issues Happen




## 3. See the Error in the Browser

https://example.com

**502 Bad Gateway**

nginx

## 4. Check Nginx Error Logs

```bash
# View recent Nginx error logs
sudo tail -n 50 /var/log/nginx/error.log

2026/10/15 10:30:21 [error] 1234#1234:
*1 connect() failed (111: Connection
refused) while connecting to upstream,
upstream: "http://127.0.0.1:8000/",
host: "example.com"
```

## 5. Check systemd Service

```bash
# Check service status
sudo systemctl status taskflow

# Start the service if it is stopped
sudo systemctl start taskflow

# View recent logs
journalctl -u taskflow -n 50
```

## 6. Check Application Logs

```bash
# View application logs (example path)
sudo tail -n 50 /var/log/taskflow/app.log

Traceback (most recent call last):
  File "/app/app.py", line 10, in <module>
    import os
ModuleNotFoundError: No module named 'os'
```

## 7. Network and Port Checks

```bash
# Check if Gunicorn is running on port 8000
sudo ss -tulnp | grep 8000

# Test locally from the server
curl -I http://127.0.0.1:8000/health

# Test Nginx configuration
sudo nginx -t

# Test external access
curl -I https://example.com
```

## 8. Failure Lab: Break and Investigate

```bash
# 1. Stop the application to simulate failure
sudo systemctl stop taskflow

# 2. Visit your domain (you should see 502)
# 3. Investigate using the steps above
# 4. Fix by starting the service
sudo systemctl start taskflow
# 5. Verify the ssrvice is running
sudo systemctl status taskflow
```

## 9. Verify and Confirm Recovery

```bash
# Check service is active
sudo systemctl status taskflow

# Test the health endpoint
curl -I http://127.0.0.1:8000/health
curl -I https://example.com/health

HTTP/1.1 200 OK
Content-Type: application/json
```

## 10. Common Issues and Fixes

| Issue | Possible Cause | Solution |
|---|---|---|
| 502 Bad Gateway | Gunicorn not running or wrong port | Start Gunicorn and check Nginx upstream config |
| Connection refused | Application not listening | Verify service and port ss -tulnp \| grep 8000 |
| Nginx 404/502 | Incorrect Nginx config | Check nginx.conf and test with nginx -t |
| Permission denied | Wrong file permissions | Set correct ownership and permissions |
| App crashes | Missing dependencies | Check application logs and install requirements |
| Timeout | Security group or network issue | Verify inbound rules and instance status |

## 11. Junior Engineer Tips

- Follow the evidence, not assumptions.
- Check each layer: browser → security group → Nginx → systemd → application.
- Use logs to find the real error.
- A 502 error usually means the application is not running or not responding.
- A healthy EC2 instance does not guarantee a healthy application.
- Always verify the health endpoint before considering the deployment fixed.

VERIQTA

# Deploy an Update and Roll Back Safely

## 1. Key Concepts

**Release Versions**
- Use version numbers to identify releases (e.g. v1.0.0, v1.1.0).
- Helps with tracking and rollback.

**Git Commits**
- Each change is a commit with a clear message.
- Push to your repository before deploying.

**Predeployment Checks**
- Check code, dependencies, and environment variables.
- Run tests if available.

**Service Restarts**
- Restart the application service after deployment.
- Verify it is running.

**Health Verification**
- Confirm the application works (e.g. health endpoint).
- Check logs for errors.

**Rollback**
- Revert to a previous working version if there are issues.
- Keep at least one stable version.

## 2. Deployment Flow (Overview)




- Make small, safe changes.
- Verify the application after deployment.
- If something goes wrong, roll back to the previous version.
- Always check logs and the health endpoint.

## 3. Update the Application (Git)

```bash
# Make your changes locally
git add .
git commit -m "Update TaskFlow to v1.1.0"
git push origin main

# SSH into your EC2 server and pull the update
ssh -i your-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
cd /opt/taskflow
git pull origin main
```

## 4. Install Dependencies and Restart Service

```bash
# Activate your virtual environment
source venv/bin/activate

# Install or update dependencies
pip install -r requirements.txt

# Restart the application service
sudo systemctl restart taskflow
sudo systemctl status taskflow
```

## 5. Verify the Deployment

```bash
# Check service status
sudo systemctl status taskflow

# Test the health endpoint
curl -I http://127.0.0.1:8000/health
curl -I https://your-domain.com/health

# Check recent logs
sudo journalctl -u taskflow -n 50
```

## 6. Roll Back to a Previous Version (If Needed)

```bash
# View commit history
git log --oneline

# Roll back to a specific commit (example)
git checkout <commit-hash>

# Or use a previous tag (example)
git checkout v1.0.0

# Reinstall dependencies if needed and restart
pip install -r requirements.txt
sudo systemctl restart taskflow
```

## 7. Practice: Deploy and Roll Back

1. Make a small change to TaskFlow (e.g. update a message).
2. Commit and push the change to GitHub.
3. Pull the update on EC2 and restart the service.
4. Verify the new version is running (check health endpoint).
5. Intentionally introduce a problem or deploy a bad change.
6. Roll back to the previous working version.
7. Confirm the application is working again.

## 8. Production Awareness

- A successful Git pull does not mean the deployment succeeded.
- Always verify the application and check logs.
- Deploy small, incremental changes.
- Have a rollback plan before deploying.
- Keep a stable, working version.
- Monitor the application after deployment.

## 9. Junior Engineer Tips

- Use clear and meaningful commit messages.
- Test changes in a non-production environment if possible.
- Always check the health endpoint.
- Check application and Nginx logs for errors.
- If something breaks, stay calm, use evidence, and roll back quickly.
- Learn from every deployment.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

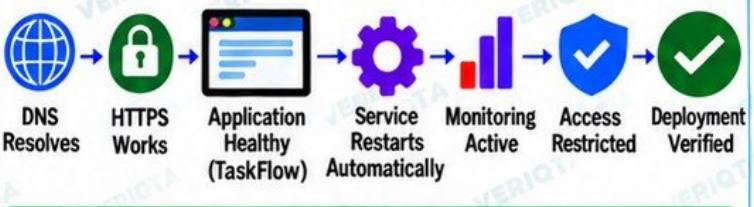
VERIQTA

# Run Your Final Deployment Verification

## 1. What to Verify

| | |
|---|---|
| DNS Resolution | Your domain resolves to the correct EC2 public IP. |
| HTTPS | Your site is accessible over HTTPS with a valid certificate. |
| Application Health | Your application is running and responds correctly. |
| Task Creation | Core features work (e.g. create a task in TaskFlow). |
| Service Restart | The application starts automatically after a restart. |
| Monitoring | CloudWatch metrics and alarms are working. |
| Access Restrictions | Only allowed traffic can access your application. |
| Recovery Steps | You can restore the application if something goes wrong. |

## 2. Deployment Verification Flow




- Confirm each step before considering the deployment complete.
- Test real application functionality (create a task).
- Verify monitoring and alerts.
- Ensure recovery steps work.
- A deployment is complete only when the expected application behavior has been verified.

## 3. Check DNS Resolution

```bash
# Check if your domain resolves to the correct IP
nslookup example.com
# or
dig example.com +short
```

## 4. Test HTTPS and Certificate

```bash
# Check HTTPS and certificate details
curl -I https://example.com
# View certificate information
openssl s_client -connect example.com:443 -servername example.com
```

## 5. Test Application Health

```bash
# Test health endpoint
curl -I https://example.com/health
curl https://example.com/health
# Expected response: 200 OK (and application details)
```

## 6. Test Core Functionality (Create a Task)

```bash
# Example using curl (adjust to your application)
curl -X POST https://example.com/api/tasks \
     -H "Content-Type: application/json" \
     -d '{"title":"Test Task","description":"Deployment check"}'
# Verify in the application UI that the task was created
```

## 7. Test Service Restart Behavior

```bash
# Restart the application service
sudo systemctl restart taskflow
# Check status
sudo systemctl status taskflow
# Test again after restart
curl https://example.com/health
```

## 8. Verify Monitoring and Access Restrictions

### Monitoring Checks

- View EC2 metrics in CloudWatch (CPU, memory, status checks).
- Confirm alarms are configured.
- Check application logs.
- Ensure you receive notifications (if configured).

### Access Restrictions

- Only ports 80 and 443 are open to the public.
- SSH (port 22) is restricted to allowed IPs.
- Application ports (e.g. 8000) are not publicly accessible.
- Test from an external network to confirm restrictions.

## 9. Final Deployment Checklist

| # | Check | Command / Action | Expected Result | Status |
|---|---|---|---|---|
| 1 | DNS resolves | nslookup example.com | Correct EC2 IP | ☑ |
| 2 | HTTPS works | curl -I https://example.com | 200 OK (valid cert) | ☑ |
| 3 | Health endpoint | curl https://example.com/health | 200 OK | ☑ |
| 4 | Create a task | Use UI or API | Task created successfully | ☑ |
| 5 | Service restart | sudo systemctl restart taskflow | Service running, app works | ☑ |
| 6 | Monitoring | Check CloudWatch | Metrics and alarms active | ☑ |
| 7 | Access restrictions | Check Security Groups | Only required ports open | ☑ |
| 8 | Recovery steps | Follow rollback/restart plan | Application restored | ☑ |

## 9. Junior Engineer Insight

- A deployment is complete only when the expected application behavior has been verified.
- Always test from a real browser and external network.
- Check both infrastructure (EC2, Nginx, systemd) and application functionality.
- Monitoring and recovery steps ar part of the deployment.
- Do not assume the deployment succeeded just because a service is running.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

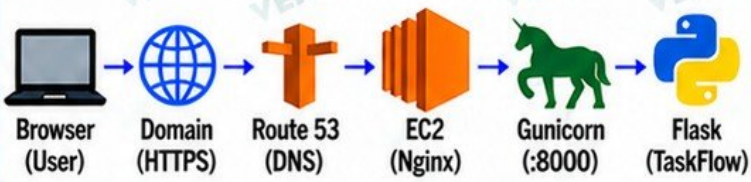
VERIQTA

# Document, Clean Up, and Present Your AWS Project

## 1. Key Concepts

| | |
|---|---|
| **Project Documentation** | Use a clear README with setup steps, architecture, screenshots, and key learnings. |
| **Architecture Diagrams** | Show how all the components connect (client → Nginx → Gunicorn → Flask on EC2). |
| **Screenshots and Evidence** | Include real screenshots of your application, terminal output, and AWS resources. |
| **Lessons Learned** | Write what went well, what was challenging, and what you would do differently. |
| **Safe Resource Cleanup** | Remove unused AWS resources to avoid unexpected charges. |
| **AWS Billing** | Check your AWS billing dashboard to confirm there are no unnecessary costs. |

## 2. Example Architecture Diagram




## 3. GitHub README Structure (Example)

```
taskflow-aws/
├── README.md
├── architecture.png
├── screenshots/
├── deployment-steps.md
├── lessons-learned.md
└── cleanup-steps.md
```

**README Should Include:**

- Project overview
- Architecture diagram
- Technologies used
- Step-by-step deployment steps
- Screenshots (application, terminal, AWS)
- How to verify the application
- Lessons learned
- Cleanup steps
- Author information

## 4. Example README Content (Snippet)

```markdown
# Taskflow - Deploy a Flask Application on AWS

## Project Overview
A simple task management web application deployed on AWS
using Nginx, Gunicorn and Flask.

## Architecture
![Architecture Diagram](architecture.png)

## How to Deploy
1. Launch an EC2 instance (Ubuntu)
2. Install and configure Nginx
3. Run Flask with Gunicorn
4. Configure a domain and HTTPS
5. Verify the deployment

## Screenshots
![Application Screenshot](screenshots/app.png)
```

## 5. Clean Up Unused AWS Resources

| | |
|---|---|
| **EC2 Instances** | Terminate instances you are no longer using. |
| **EBS Volumes** | Delete unused volumes. |
| **Elastic IPs** | Release unused Elastic IPs. |
| **Load Balancers** | Delete unused load balancers. |
| **Snapshots** | Remove old snapshots. |
| **Route 53** | Remove unused domain records (if not needed). |
| **Security Groups** | Delete unused security groups. |
| **Check AWS Billing** | Verify in the AWS Billing Dashboard that there are no unexpected charges. |

## 6. Practice: Document and Clean Up

1. Update your GitHub README with all required sections.
2. Add an architecture diagram and real screenshots.
3. Document the deployment and verification steps.
4. Write your lessons learned.
5. Remove any unused AWS resources.
6. Check your AWS billing dashboard.
7. Confirm your application still works (if keeping it).

## 7. Final Outcome

- You can clearly explain what you built and how it works.
- You have a well documented GitHub repository.
- You can demonstrate the running application.
- You know how to clean up resources and manage costs.
- You understand the full deployment process from start to finish.

## 8. Junior Engineer Insight

- A deployment is complete only when the expected application behavior has been verified.
- Documentation helps you and others understand your work.
- Cleaning up unused resources prevents unexpected charges.
- Real projects are not just about making it work, but also about showing evidence, explaining decisions, and leaving a clean environment.